

**Fundamental Architectural Invariant:** There is **NO Machine Learning (ML) model, LLM, or external cloud AI** in this codebase. 'Model' refers strictly to (1) Pydantic schemas (backend/app/models/schemas.py) and (2) deterministic mathematical/statistical formulas (Robust Z-Score via Median Absolute Deviation, Recency-Weighted Least Squares regression, Seasonal Indices) implemented with Pandas/NumPy. Every calculation is 100% auditable and reproducible.

## 0. System Mental Model & Data Flow

A user uploads a sales CSV/Excel file → Backend guesses raw column mappings ('Date', 'Item', 'Selling Price', ...) → User confirms or modifies mapping → Every row is validated, cleaned, and typed → Clean DataFrame is cached in a bounded LRU memory cache (keyed by file ID, mtime, and mapping signature) → Every dashboard view (KPI summary, charts, automated insights, inventory analytics, WLS forecast, P&L; statement, PDF report) queries **one single slicing engine** (query\_engine.build\_slice) with identical time and dimension filters → The frontend renders interactive views via Recharts, or backend streams vector PDF via ReportLab. No database is required; uploads live on disk with a configurable TTL and are swept by a background task.

## 1. Complete Technology Stack & Dependencies

| Category       | Package / Technology            | Exact Role & Implementation Detail                                                                 |
|----------------|---------------------------------|----------------------------------------------------------------------------------------------------|
| Backend Web    | FastAPI, Uvicorn, Pydantic v2   | Asynchronous REST API, strict request/response data contracts, auto OpenAPI schema docs.           |
| Numeric Math   | Pandas, NumPy, openpyxl         | 100% of data parsing, ledger math, robust statistics, WLS regression, and Excel ingestion.         |
| Document Gen   | ReportLab (Platypus)            | Server-side vector PDF generation (A4), tables, summaries, and paginated transaction ledgers.      |
| Authentication | firebase-admin, SendGrid        | Server-side JWT token verification (revocation checks), branded password-reset email dispatch.     |
| Testing (Py)   | pytest, pytest-asyncio          | 139+ tests covering edge cases, math accuracy, IDOR security, and degenerate data frames.          |
| Frontend Core  | React 18, Vite, React Router v7 | SPA client, lazy page loading, future flag compatibility, sub-second HMR dev server.               |
| State Mgmt     | Zustand (3 stores)              | useSalesStore (data & abort controllers), useThemeStore (dark/light), useDensityStore.             |
| Data Viz       | Recharts + Custom CSS Grid      | Chart Studio: 8 views (Bars, Donut, Combo, Pareto, Scatter, Treemap, Heatmap, Ranking).            |
| Styling/UI     | Tailwind CSS, Motion v12        | Utility-first responsive styles, dark mode via data-theme attribute, non-intrusive DOM animations. |

## 2. Codebase Organization & Module Directory

| File Path             | Key Functions / Classes             | Core Purpose & Mechanism                                                          |
|-----------------------|-------------------------------------|-----------------------------------------------------------------------------------|
| app/main.py           | lifespan, _sweep_loop               | FastAPI app factory, startup sweep, router registrations, CORS, /health.          |
| app/core/config.py    | Config constants, prod safety check | Loads env vars, crashes startup if DISABLE_AUTH is active in production.          |
| app/models/schemas.py | Pydantic models (Literal enums)     | Defines all API schemas with input bounds (TimeFilter, DimensionKey, MeasureKey). |

|                                    |                                                       |                                                                                    |
|------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------|
| app/api/routes/upload.py           | upload_file, confirm_mapping                          | 2-step upload: preview mapping generation → normalization and caching.             |
| app/api/routes/analytics.py        | get_summary, get_chart_data, get_insights, etc.       | Exposes all Pro and classic analytics endpoints, backed by query_engine.           |
| app/api/routes/auth.py             | forgot_password                                       | Dispatches branded password reset links via SendGrid; anti-enumeration responses.  |
| app/utils/data_validator.py        | detect_column_mapping, normalize_dataframe            | 180+ alias map + fuzzy matching, currency symbol stripping, ISO date parsing.      |
| app/services/frame_cache.py        | get_or_set, invalidate, _get_key_lock                 | Bounded LRU DataFrame cache (max 3 frames, max 120k rows) with stampede locks.     |
| app/services/query_engine.py       | build_slice, resolve_window, apply_filters, aggregate | Single chokepoint for data slicing, window bounds, multi-measure aggregation.      |
| app/services/sales_calculations.py | _prepare, compute_pnl_report, build_ledger_page       | Gross/Net revenue derivation, Chartered Accountant P&L, category ledgers.          |
| app/services/insights_engine.py    | compute_insights (6 checks)                           | MAD-based anomaly detection, top movers, margin leaks, concentration, timing.      |
| app/services/forecasting.py        | forecast_revenue, _fit_wls, _seasonal_indices         | Recency-weighted least squares (14d half-life), median seasonality, 80% intervals. |
| app/services/inventory_intel.py    | compute_inventory_intelligence                        | Demand vs stock-aware modes, ABC Pareto class, velocity, reorder priority (0-100). |
| app/services/pdf_report.py         | generate_pdf_report                                   | Platypus multi-page PDF generation with findings, P&L, forecast, and ledger.       |
| app/utils/auth_verifier.py         | get_current_user, get_firebase_app                    | Firebase ID token verification, revocation check, dev-mode bypass injection.       |

### 3 & 4. Backend Startup, Lifecycle & Configuration

#### Startup Sequence & Prod Safety Guard

- **Module Import Guard:** At import time, config.py verifies: if DISABLE\_AUTH and ENV == 'production': raise RuntimeError("DISABLE\_AUTH cannot be True in production"). Prevents accidental insecure deployments.
- **Async Lifespan:** Runs sweep\_expired\_uploads() immediately on startup to clean remnants from previous restarts, then launches \_sweep\_loop() in a background task running every UPLOAD\_SWEEP\_INTERVAL\_MINUTES (default 30 min).
- **CORS & Middleware:** Configures CORSMiddleware with ALLOWED\_ORIGINS (env-driven). Real authorization is enforced per-endpoint via Firebase ID token verification.

#### Configuration Reference (config.py)

| Constant                | Env Variable | Default          | Description & Impact                                               |
|-------------------------|--------------|------------------|--------------------------------------------------------------------|
| ALLOWED_ORIGINS         | CORS_ORIGINS | localhost:5173   | Comma-separated list of allowed frontend origins.                  |
| MAX_UPLOAD_SIZE_MB      | same         | 50               | Rejects files exceeding this size with HTTP 413.                   |
| UPLOAD_TTL_MINUTES      | same         | 120              | File retention window before automated deletion.                   |
| FRAME_CACHE_MAX_ENTRIES | same         | 3                | LRU cache capacity (tuned for 512MB RAM hosts).                    |
| FRAME_CACHE_MAX_ROWS    | same         | 120000           | DataFrames above this limit are parsed on demand and never cached. |
| FIREBASE_PROJECT_ID     | same         | senova-dashboard | Verifies Firebase token audience.                                  |

## 5 & 6. Upload Pipeline, Alias Resolution & Data Sanitization

### Two-Step Upload Flow

- **Step 1 — Preview (POST /upload/)**: Receives file, validates size ( $\leq 50\text{MB}$ ) and extension (.csv/.xlsx). Generates a 32-char hex UUID (file\_id) and writes `{file_id}{ext}` and `{file_id}.meta.json`. Sniffs delimiter and reads via utf-8-sig (falling back to latin-1 for legacy POS systems). Runs column guesser and returns 5 sample rows.
- **Step 2 — Confirmation (POST /upload/{file\_id}/confirm-mapping)**: Validates file ID regex (`^[0-9a-f]{32}$`), asserts ownership (unified 404 for security), normalizes DataFrame, writes mapping sidecar, invalidates LRU frame cache, and returns summary stats.

### Column Resolution & Math Rules (`data_validator.py`)

- **18 Canonical Fields**: 6 Required (Date, Category, Item, Quantity, Selling Price, Cost Price), 4 Optional Measures (Line Total, Discount, Tax, Stock On Hand), 8 Optional Dimensions (Branch, Payment Mode, Customer, Salesperson, Brand, Size, Colour, Invoice No).
- **Alias Resolution**: 2-tier check: (1) Exact alias map (180+ variations: 'Bill Date', 'MRP', 'Taxable Value', 'Stock Group', etc.), (2) Fuzzy keyword fallback. De-duplicates guesses so two raw columns never map to the same field.
- **Line Total Derivation Rule**: Columns like 'Amount', 'Net Amount', or 'Taxable Value' represent `Quantity × Unit Price`. They map to Line Total, and 'Selling Price' is derived as `Line Total / Quantity` (for `Quantity > 0`). This prevents massive revenue calculation errors.
- **Numeric Coercion (`_coerce_numeric`)**: Coerces via Pandas, then strips currency symbols (`₹, $, €, £, ,`) and whitespace. Maps `inf/-inf` to NaN.
- **Date Parsing (`_parse_dates`)**: Prioritizes ISO dates (`YYYY-MM-DD`) without `dayfirst` flag, then falls back to `dayfirst=True` (`DD-MM-YYYY`) for remaining rows. This prevents 2026-04-05 from flipping between April 5 and May 4.
- **Business Rules**: Drops rows with `Quantity ≤ 0`, `Selling Price < 0`, `Cost Price < 0`. Allows `Cost Price > Selling Price` (clearance sales needed for margin leak detection). Blank 'Stock On Hand' is left as NaN (never 0, which would fake stockouts).

## 7. Bounded LRU Frame Cache (`frame_cache.py`)

To prevent duplicate parsing of 50k-row Excel/CSV files across parallel API queries, the backend implements a custom thread-safe LRU cache with stampede protection:

- **Composite Cache Key**: (`file_id`, `get_file_mtime(file_id)`, `_mapping_signature(mapping)`). Any re-upload or modified column mapping automatically creates a fresh key.
- **Cache Stampede Mutex (`_get_key_lock`)**: When dashboard tabs fire 5 simultaneous requests on cold cache, threads lock on the specific key. The first thread parses and caches; remaining threads read the cached result instantly.
- **Capacity Bounds**: Default 3 entries (LRU eviction via `OrderedDict.popitem(last=False)`). Frames  $> 120,000$  rows bypass cache to prevent memory exhaustion on small 512MB RAM hosts.

## 8. Authentication & Anti-Enumeration Security

- **Firestore Token Verification (`auth_verifier.py`)**: Injected via `FastAPI Depends(get_current_user)`. Executes `firebase_auth.verify_id_token(token, check_revoked=True)` to verify signature, audience, and revocation. Returns user email or UID.
- **Dev Bypass**: When `DISABLE_AUTH=True`, returns 'dev-user@localhost' without calling Firebase. Hard-blocked from running in production.
- **Unified 404 Ownership Checks**: Malformed file ID, non-existent file, and file belonging to another user all return identical **404 File Not Found** responses. Prevents unauthorized ID enumeration.
- **Branded Password Reset (`api/routes/auth.py`)**: Generates Firebase single-use action code (1-hour validity), extracts `oobCode`, and sends a branded link via SendGrid. Always returns 200 with identical text to prevent user account enumeration.

## 9. Query Engine Slicing & Sales Ledger Math

### Single Slicing Chokepoint (`query\_engine.py`)

- **build\_slice(df, time\_filter, start\_date, end\_date, filters):** Resolves window, filters data, and extracts equal-length previous period. Used by all endpoints to guarantee 100% data consistency.
- **Window Presets:** Anchored to df.Date.max() (not server clock). Today = `[max, max+1d)`, Week = `[max-6d, max+1d)`, 30 Days = `[max-29d, max+1d)`, Month = `[1st of month, max+1d)`. Custom = `[start, end+1d)`. All presets compute an equal-length preceding period for trend comparisons.
- **Safe Filtering (apply\_filters):** Evaluates dimensions using df[col].astype(str).isin(set(values)). No dynamic string evaluation.
- **Universal Aggregator (aggregate):** Computes Revenue, Cost, Profit, Units, Transactions, Discounts, Margin %, Avg Price in one pass. Calculates Pareto 80% concentration count and folds long-tail items into `Other (N)`.

### KPIs & Ledger Calculations (`sales\_calculations.py`)

- **Net Revenue Derivation (\_prepare):** gross\_revenue = Quantity \* Selling Price; revenue = max(gross - discount, 0); cost = Quantity \* Cost Price; profit = revenue - cost.
- **Tax Isolation:** GST/Tax is tracked separately and excluded from profit ('GST collected is a liability, not income').
- **Trend Percentage:** ((current - previous) / previous) \* 100; returns 0.0 if previous is 0.
- **P&L; Statement:** Constructs formal CA-ready financial statement with Gross Sales, Discounts Allowed, Net Revenue, COGS, Gross Profit, and Tax Memo.

## 10. Detailed Mathematical Intelligence Engines

### 10.1 Automated Insights Engine (`insights\_engine.py`) — 6 Statistical Checks

- **1. Anomaly Detection (Robust Z-Score):** Computes  $z = 0.6745 * (x - \text{median}) / \text{MAD}$  where  $\text{MAD} = \text{median}(|x - \text{median}|)$ . Requires  $\geq 7$  days. If  $>25\%$  days have zero revenue, baseline switches to trading-days-only to prevent median collapsing to 0. Thresholds:  $|z| \geq 3.0$  (Critical Anomaly),  $|z| \geq 2.0$  (Warning/Spike).
- **2. Top Movers:** Evaluates items present in both current and previous windows. Ranked by **absolute rupee revenue delta** (not percentage), preventing tiny  $\blacksquare 50$  items with 300% spikes from overshadowing a  $\blacksquare 50,000$  drop.
- **3. Margin Leak:** Evaluates items contributing  $\geq 3\%$  of total revenue. Flags items whose margin is  $\geq 10.0\%$  points below the category median (or store median). Negative margin items are prioritized ( $\text{gap} = |\text{margin}| + 100$ ).
- **4. Revenue Concentration:** Evaluates Pareto 80/20 rule using `cumulative_before` threshold (requires  $\geq 8$  items). Flags if  $\leq 20\%$  of inventory drives  $\geq 80\%$  of revenue.
- **5. Timing (Weekday Variations):** Computes mean revenue per weekday (requires  $\geq 2$  observations per day). Flags if  $\text{mean\_rev}(\text{best\_day}) / \text{mean\_rev}(\text{worst\_day}) \geq 1.30$ .
- **6. Dead Stock:** Identifies items with 0 sales in the last  $\geq 30$  days (`days_since_last_sale = max_date - last_sale_date`), sorted by idle days descending.

### 10.2 Forecasting Engine (`forecasting.py`) — WLS & Seasonal Index

- **Data Guard:** Refuses forecast if history  $< 14$  days. Uses trend-only (no seasonality) for 14–21 days of data.
- **Recency-Weighted Least Squares (WLS):** Applies exponential decay weights:  $w_i = 0.5^{(\text{age\_days} / 14)}$  (14-day half-life). Fits linear trend  $y = \alpha + \beta * t$  minimizing weighted squared residuals.
- **Median Weekday Seasonality:** Computes detrended ratio  $\text{actual} / \text{trend\_fit}$ , calculates the **median ratio per weekday**, normalizes indices to mean 1.0, and clamps to  $[0.3, 3.0]$  to avoid holiday distortion.
- **Projection & Confidence Bounds:** Point forecast =  $\max(\text{trend\_t} * \text{seasonal\_idx}, 0)$ . 80% confidence interval width =  $1.2816 * \sigma_{\text{residual}} * \sqrt{1 + \text{step} / \text{history\_days}}$ .
- **Holdout Backtesting (7 Days):** Measures accuracy on held-out 7 days. Normal series: Accuracy =  $\text{clamp}(100 - \text{MAPE}, 0, 100)$  where  $\text{MAPE} = \text{mean}(|\text{actual} - \text{pred}| / \text{actual}) * 100$ . Sparse series ( $>25\%$  zero days): scores total volume accuracy  $100 - |\text{sum}(\text{pred}) - \text{sum}(\text{act})| / \text{sum}(\text{act}) * 100$ .

10.3 Inventory Intelligence (`inventory_intel.py`)

- **Dual Operational Modes:** Demand Mode (sales only) vs Stock-Aware Mode (when `Stock On Hand` is provided).
- **Velocity:** `velocity_per_day` = `units / window_days` and `velocity_active_days` = `units / active_days`.
- **ABC Classification:** A (Top 80% cumulative revenue), B (Next 15% revenue), C (Remaining 5% revenue).
- **Stock Ageing Buckets:** Fresh (<15 days idle), Slow (15–29 days), Stale (30–59 days), Dead (≥60 days idle).
- **Reorder Priority Score (0–100):**  $100 * (0.50 * velocity\_norm + 0.30 * trend\_norm + 0.20 * recency\_norm)$ .
- **Stock-Aware Metrics:** `Days of Cover` = `stock_on_hand / velocity_per_day`; `Capital Locked` = `stock_on_hand * avg_cost_price`; `Reorder Flag` = `days_of_cover < 7 days`.

11. Server-Side PDF Report Generator (`pdf_report.py`)

Generates clean, professional A4 vector PDF documents using ReportLab Platypus. Every metric is computed from the same `build_slice` call, ensuring 100% visual parity with the screen:

- **1. Header & Summary:** Report title, original filename, active time filter, and date range.
- **2. Automated Findings:** Insights printed first with text-based severity badges (`URGENT`, `WATCH`, `GOOD`, `NOTE`) for photocopy readability.
- **3. Financial P&L; Statement:** Formal financial layout with Gross Sales, Discounts, Net Revenue, COGS, Gross Profit, and Tax Liability Memo.
- **4. Category Ledger & Forecast:** Revenue breakdown by category and 7–30 day forward forecast.
- **5. Inventory Reorder & Dead Stock:** Top 20 items ranked by Reorder Priority and dead stock table.
- **6. Paginated Transaction Ledger:** Capped at 500 rows (`MAX_LEDGER_ROWS_IN_PDF`) with overflow notification.

12. Complete Backend REST API Reference

| Method & Endpoint                 | Request Body / Params | Response Model        | Functional Description                                                   |
|-----------------------------------|-----------------------|-----------------------|--------------------------------------------------------------------------|
| POST /upload/                     | multipart file        | ColumnMappingPreview  | Uploads file, sniffs delimiter, returns preview & 5 sample rows.         |
| POST /upload/{id}/confirm-mapping | {mapping: dict}       | UploadResponse        | Validates rows, saves mapping sidecar, primes LRU frame cache.           |
| POST /analytics/{id}/summary      | AnalysisQuery         | AnalyticsResponse     | KPI summary (Revenue, Profit, Margin %, Units, Trends, Top Items).       |
| POST /analytics/{id}/chart-data   | ChartQuery            | ChartDataResponse     | Universal dimension aggregator with Pareto 80% and long-tail folding.    |
| POST /analytics/{id}/heatmap      | ChartQuery            | HeatmapResponse       | Weekday x ISO-Week matrix with numeric min/max bounds.                   |
| POST /analytics/{id}/insights     | AnalysisQuery         | InsightsResponse      | Runs all 6 statistical finding checks and returns prioritized cards.     |
| POST /analytics/{id}/forecast     | ForecastQuery         | ForecastResponse      | WLS trend projection, confidence intervals, MAPE backtest score.         |
| POST /analytics/{id}/inventory    | AnalysisQuery         | InventoryResponse     | Velocity, ABC classification, ageing buckets, reorder priority scores.   |
| POST /analytics/{id}/report       | AnalysisQuery         | CAReportSummary       | Chartered Accountant P&L; statement, revenue breakdown, category ledger. |
| POST /analytics/{id}/report.pdf   | AnalysisQuery         | application/pdf bytes | Generates and streams downloadable ReportLab PDF file.                   |

|                             |              |                        |                                                                    |
|-----------------------------|--------------|------------------------|--------------------------------------------------------------------|
| POST /analytics/{id}/ledger | LedgerQuery  | LedgerPage             | Paginated, sorted raw transaction ledger with date/column filters. |
| POST /auth/forgot-password  | {email: str} | ForgotPasswordResponse | Generates Firebase action code, sends branded email via SendGrid.  |
| GET /health                 | None         | {status: 'ok'}         | Public health check endpoint for deployment monitoring.            |

### 13. Frontend Architecture & State Management

- **Zustand Stores:** useSalesStore manages global query state ({timeFilter, startDate, endDate, filters}), upload state, and cancellable fetchers with AbortController to prevent race conditions on fast filter toggles. useThemeStore and useDensityStore control UI theme and compact layout.
- **URL as Source of Truth:** Dashboard.jsx synchronizes state with URL search params (range, from, to, filters). Allows users to share exact filtered views with accountants.
- **Chart Studio (8 Views, 1 Payload):** chartView.js turns view configurations into API requests. Because query\_engine.aggregate precomputes all measures in a single response, toggling chart views (Bar, Donut, Pareto, Ranking, etc.) switches components instantly without extra network round trips.
- **Drill-Down Capabilities:** Clicking any chart element or table row opens DrillDownPanel, fetching filtered transaction lines directly from /ledger.

### 14. Test Suite Coverage & Verification

The backend test suite consists of **139 test functions across 10 test files** in backend/tests/:

- test\_data\_validator.py: 180+ alias maps, fuzzy resolution, Line Total unit price derivation, numeric cleaning, ISO date parsing.
- test\_query\_engine.py: Window resolution (presets anchored to max date), safe filtering, Pareto concentration, heatmap matrix.
- test\_insights\_engine.py: All 6 statistical checks, MAD anomaly detection, single-item catalog edge cases.
- test\_forecasting.py: WLS slope/intercept, 14-day history guard, median weekday seasonality, 80% confidence bands, MAPE backtesting.
- test\_inventory\_and\_forecast.py: ABC cumulative revenue threshold, velocity metrics, reorder priority scoring.
- test\_api.py: Complete upload → confirm → analytics workflow, IDOR ownership verification, 404 security checks.
- test\_accuracy\_audit.py: Independent re-computation audit comparing engine outputs against raw unoptimized Pandas formulas.
- test\_edge\_cases.py: Degenerate inputs (1-row files, all-zero revenue, infinite values, negative margins).

### 15. Known Documentation Discrepancies (Code vs Docs)

Audited discrepancies between existing markdown docs and actual codebase implementation:

- **1. Test Count Drift:** README notes 132 tests; direct source count has 139 test functions; older UI doc notes 177. (Caused by pytest parametrization expansion and test suite growth).
- **2. Auth Routes Undocumented:** app/api/routes/auth.py and email\_service.py are fully implemented with 19 tests, but missing from earlier architecture markdown documents.
- **3. Design Proposal vs Code:** docs/UPGRADES.md describes WhatsApp sharing and persistent database history as future proposals — they are design specs and not implemented.

### 16. System Constants & Mathematical Parameters Reference

| Constant / Parameter | Value | File Location | Exact Mathematical Purpose |
|----------------------|-------|---------------|----------------------------|
|----------------------|-------|---------------|----------------------------|

|                          |                      |                       |                                                                                                                            |
|--------------------------|----------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------|
| MAD_TO_SIGMA_SCALE       | 0.6745               | insights_engine.py    | Scales Median Absolute Deviation to normal distribution standard deviation (1 / 1.4826).                                   |
| ANOMALY_THRESHOLDS       | 3.0 / 2.0            | insights_engine.py    | Z-score thresholds for Critical Anomaly ( $z \geq 3.0$ ) and Warning Spike/Drop ( $z \geq 2.0$ ).                          |
| SPARSE_ZERO_THRESHOLD    | 0.25 (25%)           | insights, forecasting | Switches to trading-days-only stats if >25% calendar days have zero revenue.                                               |
| MARGIN_LEAK_THRESHOLDS   | 3% rev, 10% gap      | insights_engine.py    | Flags items with $\geq 3\%$ revenue share whose margin is $\geq 10.0\%$ pts below median.                                  |
| CONCENTRATION_THRESHOLDS | 20% items, 80% rev   | insights_engine.py    | Flags concentration if $\leq 20\%$ of catalog items generate $\geq 80\%$ of total revenue.                                 |
| WEEKDAY_RATIO_THRESHOLD  | 1.30 (30%)           | insights_engine.py    | Flags timing insight if best weekday average revenue is $\geq 30\%$ higher than worst.                                     |
| DEAD_STOCK_DAYS          | 30 days              | insights, sales_calc  | Threshold for dead stock inactivity.                                                                                       |
| FORECAST_MIN_HISTORY     | 14 days              | forecasting.py        | Refuses forecast below 14 days; requires 21 days for weekday seasonality.                                                  |
| WLS_HALF_LIFE            | 14 days              | forecasting.py        | Exponential weight decay half-life: $w = 0.5^{**}(\text{age} / 14)$ .                                                      |
| SEASONAL_INDEX_CLAMP     | [0.3, 3.0]           | forecasting.py        | Bounds seasonal multiplier to prevent extreme single-day holiday distortion.                                               |
| CONFIDENCE_Z_80PCT       | 1.2816               | forecasting.py        | Z-multiplier for 80% two-sided forecast prediction interval.                                                               |
| BACKTEST_HOLDOUT_DAYS    | 7 days               | forecasting.py        | Holdout period used to evaluate forecast MAPE accuracy score.                                                              |
| ABC_BOUNDARIES           | 80% / 95%            | inventory_intel.py    | Cumulative revenue share split: Class A ( $< 80\%$ ), Class B ( $< 95\%$ ), Class C (rest).                                |
| AGEING_BUCKETS           | 15 / 30 / 60 days    | inventory_intel.py    | Inventory status: Fresh ( $< 15\text{d}$ ), Slow ( $< 30\text{d}$ ), Stale ( $< 60\text{d}$ ), Dead ( $\geq 60\text{d}$ ). |
| REORDER_WEIGHTS          | 0.50 / 0.30 / 0.20   | inventory_intel.py    | Weights for Reorder Score: Velocity (50%), Trend (30%), Recency (20%).                                                     |
| REORDER_COVER_DAYS       | 7 days               | inventory_intel.py    | Days of cover threshold ( $< 7$ days triggers reorder flag).                                                               |
| MAX_UPLOAD_SIZE_MB       | 50 MB                | config.py             | Maximum allowable file upload size.                                                                                        |
| UPLOAD_TTL_MINUTES       | 120 min              | config.py             | File retention window on disk before background task purge.                                                                |
| FRAME_CACHE_LIMITS       | 3 frames / 120k rows | config.py             | Memory bounds for in-memory LRU DataFrame cache.                                                                           |
| MAX_LEDGER_ROWS_IN_PDF   | 500 rows             | pdf_report.py         | Maximum raw transaction rows rendered into the exported PDF ledger.                                                        |